

Intro to Software Engineering – Quiz #4

July 12, 2024

Quiz #4
Total: 100 points
Due Date: July 26th, 2024

*This quiz must be completed individually.
I.e. you should work alone without any help or collaboration from others.
Please provide detailed answers to the following exercises.*

Question 1. Spot the mistakes [4 marks per mistake identified, **20 total**]

Given the following REST API below, find all the REST API design mistakes.

You may assume that there are no syntax errors, all methods used by the endpoints are defined, and a user is logged in before every request

As a hint, there are exactly **5** mistakes.

For each mistake identify 3 things:

- Which endpoint/area of the code the mistake occurs
- A description of the mistake
- Describe how you'd fix the mistake, don't provide any code, just describe in words

(Note below is from a single Web API file. It's been broken into different images for readability)

```
let totalPosts = 0;
app.get("/api/users/:imageId", (req, res)=>{
  if(ImageDatabase.getImageById(req.params.imageId) == null){
    return res.status(404).json({message: "Image not found"});
  }

  let image = ImageDatabase.getImageById(req.params.imageId);
  return res.json(image);
});

app.get("/api/users/:userId", (req, res) => {
  let currentUser = req.params.userId;
  let user = UserDatabase.getUserById(currentUser);
  if(!user)
    return res.status(404).json({message: "User not found"});
  return res.json(user);
})
```

```
app.get("/api/users/logout", (req, res) => {
  // Logout the user by removing them from currently logged
  // in table in our database
  UserDatabase.logoutUser(req.body.userId);
  return res.json({message: "Logged out"});
})

app.get("/api/users/images", (req, res) => {
  let currentUser = req.body.userId;
  let user = UserDatabase.getUserById(currentUser);
  if(!user)
    return res.status(404).json({message: "User not found"});
  let userImages = UserDatabase.getImagesByUser(currentUser);
  return res.json(userImages);
})

app.post("/api/users/:userId/images", (req, res) => {
  let currentUser = req.params.userId;
  let user = UserDatabase.getUserById(currentUser);
  let {title, imageFile, description} = req.body;
  if(!user)
    return res.status(404).json({message: "User not found"});
  try{
    UserDatabase.addImageForUser(currentUser, title, imageFile, description);
    totalPosts++;
  } catch (error) {
    if(error.type == "SQL_CONNECTION_ERROR")
      return res.status(400).json({message: "Could not add image"});
    return res.status(500).json({message: "Could not add image"});
  }
  return res.json({message: "Image added"});
});

app.get("/api/images/totalPosts", (req, res) => {
  return res.json({totalPosts});
})
```

Question 2. Documenting Web APIs [5 per API, 20 total]

For your project, provide an API specification for exactly four out of the following five HTTP methods:

- GET
- POST
- PATCH
- DELETE
- PUT

If you don't have four instances of these methods, you must provide reasonable justification behind why your application doesn't contain one of these methods, or what alternative methods your group used.

Example:

Path: /api/books/:id/purchase

Method: PATCH

Body:

- Amount
 - integer
 - Description: The amount to purchase

Description: Purchases some books based on the amount provided in the body

Responses:

- 200
 - Description: Purchase was successful

```
{
  "book": {
    "title": "CSCC01 for Dummies",
    "author": "Pankaj Agrawal",
    "price": 20,
    "stock": 99
  },
  "cost": 20,
  "units_purchased": 1
}
```

- 500
 - Description: Server error, couldn't purchase book

```
{
  "message": "server error, could not complete transaction"
}
```

- 422

- Description: Invalid arguments provided. Likely the amount was negative

```
{  
  "message": "Please provide a non-negative amount"  
}
```

- 409

- Description: Not enough books in stock to fulfill order

```
{  
  "message": "Not enough books in stock to fulfill this order"  
}
```

Question 3 Web API Performance

[8 for correctly identifying problem

8 for correct solution proposal

4 for correct pseudocode

20 total]

Consider the following endpoint, and pseudocode:

```
GET /api/images/  
function () {  
    all_images = db.images.findAll();  
    return res.status(200).json(all_images);  
}
```

- Describe a scenario where running this endpoint could be harmful.
- Propose a solution to address this issue.
- Provide pseudocode demonstrating your proposed solution alternative behavior/strategy for this endpoint.

Feel free to be as abstract as you want with your pseudocode, as long as what you propose is realistic.

Question 4 Web API Implementation [5 per API, 40 total]

You will be implementing a REST API for managing university courses, and students.

Step 1: Download, and extract the startcode (**c01_rest_quiz.zip**)

You will be using Express, and Sequelize (an ORM) with SQLite.

You don't need prior knowledge of Sequelize. A class with all necessary functions is provided in **"/db_utils/db.js"**. Feel free to add or modify methods in this file or directly use Sequelize.

Step 2: Find the /me endpoint in the index.js file, and insert your utorid in the response. **This step is very important!!**

Step 3: Implement all of the following APIs

Note:

- The API specification includes the expected success responses but does not specify error responses. It is your responsibility to determine appropriate error responses and when to return them. Ensure extensive error handling for every endpoint.
- Some API specifications are missing a method (denoted by "?"). It is your job to decide which appropriate method will belong to the API.
- This portion will be heavily autograded, therefore it is very important that you follow the API naming conventions exactly!
- For the sample responses, providing extra fields is **OK**.

API Specification**Add a course**

URL: **"/api/courses"**

Method: POST

Request Body:

```
{
  "code": "courseCode",
  "name": "course name",
  "session": "course session (eg: F23, W24)",
  "department": "department (eg: CSC, BIO, ENG)"
}
```

Sample 200 Response:

```
{
  "course": {
    "id": 1,
    "code": "CSCA08",
    "name": "Introduction to Programming",
    "session": "F24",
    "department": "CSC"
  }
}
```

Get courses

URL: `"/api/courses"`

Method: GET

Query Parameters:

- `"department"` (optional, string): Filter courses by department
- `"session"` (optional, string): Filter courses by session

Sample 200 Response:

```
{
  "courses": [
    {
      "id": 1,
      "code": "CSCA08",
      "name": "Introduction to Programming",
      "session": "F24",
      "department": "CSC"
    },
    ...
  ]
}
```

Get students enrolled in a course

URL: `"/api/courses/{courseId}/students"`

Method: GET

Path Parameters:

- `"courseId"` (string) - The course enrollment to retrieve

Sample 200 Response (**Note:** Your response may look different. As long as you return the student's names, id, courseId, and finalGrade in some form, it is **OK**)

```
{
  "students": [
    {
      "id": 1,
      "finalGrade": 93,
      "createdAt": "2024-07-08T22:36:53.563Z",
      "updatedAt": "2024-07-08T22:47:37.736Z",
      "CourseId": 1,
      "StudentId": 1,
      "Student": {
        "id": 1,
        "firstName": "Porom",
        "lastName": "Kamal",
        "phoneNumber": "1234567890",
        "createdAt": "2024-07-08T22:36:40.960Z",
        "updatedAt": "2024-07-08T22:36:40.960Z"
      }
    },
    ...
  ]
}
```

Add student to a course

URL: `"/api/courses/{courseId}/students"`

Method: POST

Path Parameter:

- `"courseId"` (integer) - The course Id to enroll the student into

Request Body:

```
{
  "studentId": INTEGER
}
```

Sample 200 Response:

```
{
  "enrolment": {
    "id": 5,
    "courseId": 1,
    "studentId": 123,
  }
}
```


Update a student's grade in a courseURL: `"/api/course/{courseId}/students/{studentId}/grade`Method: `?`

Request Body:

```
{
  "finalGrade": INTEGER
}
```

Sample 200 Response:

```
{
  "enrolment": {
    "courseId": 1,
    "studentId": 123,
    "finalGrade": 93
  }
}
```

Add a studentURL: `"/api/students"`Method: `?`

Request Body:

```
{
  "firstName": "Jane",
  "lastName": "Smith",
  "phoneNumber": "987-654-3210"
}
```

Sample 200 Response:

```
{
  "student": {
    "id": 1,
    "firstName": "Jane",
    "lastName": "Smith",
    "phoneNumber": "987-654-3210"
  }
}
```

Update a studentURL: `"/api/students"`Method: `?`

Request Body:

```
{
  "firstName": "Jane",
  "lastName": "Smith",
  "phoneNumber": "987-654-3210"
}
```

Sample 200 Response:

```
{
  "student": {
    "id": 1,
    "firstName": "Jane",
    "lastName": "Smith",
    "phoneNumber": "987-654-3210"
  }
}
```

Delete a studentURL: `"/api/students/{studentId}"`Method: `?`

Sample 200 Response:

```
{
  "student": {
    "id": 1,
    "firstName": "Jane",
    "lastName": "Smith",
    "phoneNumber": "987-654-3210"
  }
}
```

Step 4: Prepare for Submission

Make sure to do the following before the final submission.

- Delete the `node_modules` folder. **Any submissions which still contain this folder will be discarded!**
- Delete the `microblog.sqlite` file, otherwise this may interfere with the autograder
- Zip your solution, and name it `"YOUR_UTORID-quiz_4.zip"`

Submission

You will need to upload 2 artifacts to Quercus:

1. A PDF containing your answers to the written questions
2. A zip file containing all your code from question 4